

Manual final completo de la practica

PostGIS con psycopg, Django, Docker y pgAdmin

Proyecto campus: puntos, lineas y poligonos

Objetivo: Iniciar desde cero, ordenar el workspace, crear las bases, programar todos los scripts y dejar el proyecto listo para urls.py y views.py.

Alcance de esta version: Incluye todos los codigos completos: SQL, myLib, psycampus OOP, modelos Django, djcampus, urls.py, views.py, pruebas, errores y checklist final.

Referencia: Se toma como base tu manual corregido y se amplian los archivos que antes quedaron resumidos, especialmente lineasOOP.py, poligonosOOP.py y updatedj.py.

Indice

1. Arquitectura general del proyecto
2. Estructura final de carpetas
3. Donde se ejecuta cada cosa
4. Preparacion limpia del workspace
5. Bases de datos en PostGIS
6. Parte A - psycopg con SQL directo
7. Parte B - Django con modelos y scripts
8. Parte C - URLs y views
9. Pruebas por consola, navegador y Postman
10. Que hacer si ya esta generado
11. Relacion con scripts antiguos de scriptsBase.rar
12. Errores frecuentes
13. Comandos finales y checklist

1. Arquitectura general del proyecto

La practica separa dos formas de trabajar con PostGIS. La primera usa psycopg y SQL directo; la segunda usa Django ORM, modelos, migraciones y scripts ejecutados con manage.py runscript. Ambas trabajan con las mismas capas conceptuales, pero no deben mezclarse en la misma base durante las pruebas iniciales.

Componente	Funcion	Base de datos
psycopg	Ejecuta SQL directo desde Python. Permite practicar consultas PostGIS y validaciones espaciales.	proyecto_psy
Django	Usa modelos, migraciones y scripts. Luego se conecta a urls.py y views.py.	proyecto_desweb
PostGIS	Extension espacial de PostgreSQL. Guarda geometry(Point, LineString, Polygon).	Ambas
pgAdmin	Interfaz grafica para revisar tablas, columnas, registros y SQL.	Ambas

Idea clave: No copies buildings2 como entrega final. buildings2 sirve como ejemplo de rutas y vistas; tu proyecto final se centra en campus.

Flujo general

Windows / PowerShell

- > docker compose up
- > docker ps
- > docker compose down

Attach Shell del contenedor djangoapi

- > cd /usr/src/app
- > python manage.py check
- > python manage.py runscript ...
- > cd /usr/src/app/scripts/p1
- > python main.py ...

Attach Shell del contenedor postgis

- > psql -U postgres
- > CREATE DATABASE
- > CREATE EXTENSION postgis
- > ejecutar SQL de tablas

pgAdmin en navegador

- > revisar proyecto_psy y proyecto_desweb
- > abrir Query Tool
- > verificar tablas y registros

Navegador / Postman

- > probar endpoints /campus/... despues de crear urls.py y views.py

- PostGIS agrega tipos y funciones espaciales: `geometry(Point)`, `geometry(LineString)`, `geometry(Polygon)`, `ST_Within`, `ST_Relate`, `ST_Area` y `ST_Length`.
- EPSG:25830 corresponde a ETRS89 / UTM zona 30N. Sus unidades son metros, por eso longitud, area y perimetro son interpretables directamente.
- WKT significa Well-Known Text. Ejemplos: `POINT(20 20)`, `LINESTRING(10 10, 20 20)`, `POLYGON((0 0, 100 0, 100 100, 0 0))`.
- `ST_SnapToGrid` redondea coordenadas a una malla. En esta practica se usa 0.0001 para reducir ruido numerico.
- Django ORM representa tablas como clases Python. Las migraciones transforman esas clases en tablas fisicas dentro de PostgreSQL/PostGIS.

2. Estructura final de carpetas

Esta estructura evita mezclar ejemplos del profesor con tu proyecto campus. Los archivos activos deben estar en `campus` y `scripts/p1`.

```
C:
├── proyecto_desweb
│   ├── docker-compose.yml
│   ├── .env
│   ├── .env.dev
│   └── djangoapi\
│       ├── manage.py
│       └── djangoapi\
│           ├── __init__.py
│           ├── settings.py
│           ├── urls.py
│           ├── asgi.py
│           └── wsgi.py
│       └── campus\
│           ├── __init__.py
│           ├── admin.py
│           ├── apps.py
│           ├── models.py
│           ├── urls.py
│           ├── views.py
│           └── migrations\
│               └── __init__.py
│       └── scripts\
│           └── p1\
│               ├── __init__.py
│               ├── main.py
│               ├── 00_create_db.sql
│               ├── 01_schema.sql
│               └── myLib\
│                   ├── __init__.py
│                   ├── p1Settings.py
│                   └── connect.py
│               └── psycampus\
│                   ├── __init__.py
│                   ├── puntosOOP.py
│                   ├── lineasOOP.py
│                   └── poligonosOOP.py
│               └── djcampus\
│                   ├── __init__.py
│                   ├── insertdj.py
│                   ├── selectdj.py
│                   ├── updatedj.py
│                   └── deletedj.py
```

- `scripts/p1/buildings` y `scripts/p1/buildingsDjangoModels` no deben quedar como codigo activo de tu entrega campus.
- `main copy.py`, `main copy 2.py`, `mainDjango.py` y `mainOOP.py` pueden guardarse como respaldo, pero no deben ser el flujo oficial.
- Cualquier archivo con `from tu_app.models` debe corregirse o descartarse.
- Si `djangoapi/urls.py` incluye `buildings2.urls` y no tienes esa app completa, Django puede romper al iniciar.

3. Donde se ejecuta cada cosa

Lugar	Uso	Comandos tipicos
PowerShell en Windows	Levantar/detener Docker, ver contenedores y ordenar carpetas.	docker compose up, docker compose down, docker ps, dir, ren
Attach Shell de djangoapi	Ejecutar manage.py, scripts Django y scripts psycopg.	python manage.py check, python manage.py runscript, python main.py
Attach Shell de postgis	Entrar a psql, crear bases, activar PostGIS y ejecutar SQL.	psql -U postgres, CREATE DATABASE, CREATE EXTENSION
pgAdmin	Revisar graficamente bases, tablas, columnas y datos.	Query Tool, Browse, Refresh
Navegador/Postman	Probar endpoints despues de configurar urls.py y views.py.	GET /campus/hello/, POST /campus/poligonos/insert/

Aclaracion: El contenedor pgAdmin normalmente no se usa por shell. Se usa desde navegador. El shell importante para SQL es el de postgis.

4. Preparacion limpia del workspace

Si confundiste el workspace de la plantilla del profesor con tu proyecto, no borres nada. Primero renombra la carpeta mezclada y trabaja desde una copia limpia.

```
# PowerShell, desde C:\
cd C:\

# Guardar el proyecto mezclado como respaldo
ren proyecto_desweb proyecto_desweb_respaldo_confundido

# Extraer o copiar el backup limpio y renombrarlo como proyecto_desweb
# Debe quedar asi:
# C:\proyecto_desweb

cd C:\proyecto_desweb
dir

# Si solo esta mezclada la carpeta p1
cd C:\proyecto_desweb\djangoapi\scripts
ren p1 p1_respaldo_desordenado
mkdir p1

# Luego crea manualmente:
# p1\myLib
# p1\psycampus
# p1\djcampus
# p1\__init__.py
# p1\main.py
# p1\00_create_db.sql
# p1\01_schema.sql
```

5. Bases de datos en PostGIS

Base	Uso	Tablas esperadas
proyecto_psy	Parte psycopg con SQL directo.	campus_puntos, campus_lineas, campus_poligonos
proyecto_desweb	Parte Django ORM, migraciones, urls.py y views.py.	campus_puntos, campus_lineas, campus_poligonos + django_* + auth_*

5.1 Crear bases desde cero en postgis

```
# PowerShell: ver contenedores
cd C:\proyecto_desweb
docker compose up

# En otra terminal PowerShell
docker ps

# Entrar al contenedor postgis
```

```

docker exec -it NOMBRE_CONTENEDOR_POSTGIS /bin/sh

# Entrar a PostgreSQL
psql -U postgres

-- Dentro de psql
CREATE DATABASE proyecto_psy;
CREATE DATABASE proyecto_desweb;

\c proyecto_psy
CREATE EXTENSION IF NOT EXISTS postgis;

\c proyecto_desweb
CREATE EXTENSION IF NOT EXISTS postgis;

\l
\q

```

5.2 00_create_db.sql

C:\proyecto_desweb\djangoapi\scripts\p1\00_create_db.sql

```

-- 00_create_db.sql
-- Uso: crear las dos bases si estas iniciando desde cero.
-- Si alguna base ya existe, PostgreSQL mostrara error; en ese caso no repitas este archivo.

CREATE DATABASE proyecto_psy;
CREATE DATABASE proyecto_desweb;

```

5.3 01_schema.sql para proyecto_psy

C:\proyecto_desweb\djangoapi\scripts\p1\01_schema.sql

```

-- 01_schema.sql
-- Este esquema se usa para la base proyecto_psy, que trabaja con psycopg y SQL directo.
-- En proyecto_desweb, las tablas de campus deben crearse preferentemente con migraciones de Django.

DROP TABLE IF EXISTS campus_puntos CASCADE;
CREATE TABLE campus_puntos (
    id SERIAL PRIMARY KEY,
    tipo VARCHAR(50) NOT NULL,
    estado VARCHAR(50) NOT NULL,
    material VARCHAR(50),
    codigo_inventario VARCHAR(20) UNIQUE,
    observacion TEXT,
    geom geometry(Point, 25830) NOT NULL
);
CREATE INDEX campus_puntos_gix ON campus_puntos USING GIST (geom);

DROP TABLE IF EXISTS campus_lineas CASCADE;
CREATE TABLE campus_lineas (
    id SERIAL PRIMARY KEY,
    tipo_via VARCHAR(50) NOT NULL,
    pavimento VARCHAR(50),
    accesible BOOLEAN DEFAULT TRUE,
    codigo_tramo VARCHAR(20) UNIQUE,
    observacion TEXT,
    geom geometry(LineString, 25830) NOT NULL,
    longitud DOUBLE PRECISION GENERATED ALWAYS AS (ST_Length(geom)) STORED
);
CREATE INDEX campus_lineas_gix ON campus_lineas USING GIST (geom);

DROP TABLE IF EXISTS campus_poligonos CASCADE;
CREATE TABLE campus_poligonos (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    uso_principal VARCHAR(50),
    pisos INTEGER,
    estado VARCHAR(50),
    observacion TEXT,
    geom geometry(Polygon, 25830) NOT NULL,
    area DOUBLE PRECISION GENERATED ALWAYS AS (ST_Area(geom)) STORED,
    perimetro DOUBLE PRECISION GENERATED ALWAYS AS (ST_Perimeter(geom)) STORED
);
CREATE INDEX campus_poligonos_gix ON campus_poligonos USING GIST (geom);

```

5.4 Ejecutar el esquema en proyecto_psy

```
# Opcion A: desde Attach Shell del contenedor djangoapi
cd /usr/src/app
psql -h postgres -U postgres -d proyecto_psy -f /usr/src/app/scripts/p1/01_schema.sql

# Comprobar tablas
psql -h postgres -U postgres -d proyecto_psy
\dt
\q

# Opcion B: desde PowerShell, enviando el SQL al contenedor postgres
cd C:\proyecto_desweb
Get-Content .\djangoapi\scripts\p1\01_schema.sql -Raw | docker exec -i NOMBRE_CONTENEDOR_POSTGIS psql -U
postgres -d proyecto_psy

# Comprobar tablas
docker exec -it NOMBRE_CONTENEDOR_POSTGIS psql -U postgres -d proyecto_psy
\dt
\q
```

pgAdmin: Tambien puedes abrir proyecto_psy > Query Tool, pegar 01_schema.sql y ejecutar. Luego refresca Schemas > public > Tables.

6. Parte A - psycopg con SQL directo

Esta parte no usa Django ORM. Abre una conexion psycopg hacia proyecto_psy y ejecuta SQL directo contra PostGIS.

6.1 p1Settings.py

C:\proyecto_desweb\djangoapi\scripts\p1\myLib\p1Settings.py

```
# scripts/p1/myLib/p1Settings.py
# Configuracion comun para la practica p1.
# La parte psycopg usa estos datos para conectarse a proyecto_psy.
# La parte Django solo reutiliza EPSG_CODE y SNAP_DISTANCE; Django se conecta usando settings.py.

import os

POSTGRES_USER = os.getenv("POSTGRES_USER", "postgres")
POSTGRES_PASSWORD = os.getenv("POSTGRES_PASSWORD", "postgres")

# Usamos una variable propia para evitar confundirla con POSTGRES_DB del proyecto Django.
POSTGRES_DB = os.getenv("P1_POSTGRES_DB", "proyecto_psy")

# Dentro de Docker, el host debe ser el nombre del servicio de base de datos.
# Normalmente se llama postgres en docker-compose.yml.
POSTGRES_HOST = os.getenv("POSTGRES_HOST", "postgres")
POSTGRES_PORT = os.getenv("POSTGRES_PORT", "5432")

EPSG_CODE = 25830
SNAP_DISTANCE = 0.0001
```

6.2 connect.py

C:\proyecto_desweb\djangoapi\scripts\p1\myLib\connect.py

```
# scripts/p1/myLib/connect.py
# Abre una conexion psycopg hacia la base definida en p1Settings.py.
# Este archivo se usa solo para la parte psycopg, no para Django ORM.

import psycopg
from myLib import p1Settings

def connect():
    conn = psycopg.connect(
        dbname=p1Settings.POSTGRES_DB,
        user=p1Settings.POSTGRES_USER,
        password=p1Settings.POSTGRES_PASSWORD,
        host=p1Settings.POSTGRES_HOST,
        port=p1Settings.POSTGRES_PORT,
```

```
)  
return conn
```

6.3 puntosOOP.py

C:\proyecto_desweb\djangoapi\scripts\p1\psycampus\puntosOOP.py

```
# scripts/p1/psycampus/puntosOOP.py  
# CRUD de campus_puntos usando psycopg y SQL directo.  
# Regla espacial: el punto debe estar dentro de algun poligono existente.  
  
from psycopg.rows import dict_row  
from myLib.connect import connect  
from myLib.p1Settings import EPSG_CODE, SNAP_DISTANCE  
  
class PuntosOOP:  
    def __init__(self):  
        self.conn = connect()  
        self.cur = self.conn.cursor()  
  
    def disconnect(self):  
        self.cur.close()  
        self.conn.close()  
  
    def _result(self, ok, message, data):  
        return {"ok": ok, "message": message, "data": data}  
  
    def _check_keys(self, d, keys):  
        for key in keys:  
            if key not in d:  
                raise Exception(f"Falta el campo: {key}")  
  
    def _is_valid_geom(self, geom_wkt):  
        self.cur.execute(  
            """  
            SELECT ST_IsValid(  
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)  
            )  
            """,  
            [geom_wkt, EPSG_CODE, SNAP_DISTANCE],  
        )  
        return self.cur.fetchone()[0]  
  
    def _point_inside_polygon(self, geom_wkt):  
        self.cur.execute(  
            """  
            SELECT id  
            FROM campus_poligonos  
            WHERE ST_Within(  
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),  
                geom  
            )  
            LIMIT 1  
            """,  
            [geom_wkt, EPSG_CODE, SNAP_DISTANCE],  
        )  
        return len(self.cur.fetchall()) > 0  
  
    def insert(self, d):  
        try:  
            self._check_keys(  
                d,  
                ["tipo", "estado", "material", "codigo_inventario", "observacion", "geom"],  
            )  
            geom = d["geom"]  
  
            if not self._is_valid_geom(geom):  
                self.disconnect()  
                return self._result(False, "La geometria del punto no es valida", [])  
  
            if not self._point_inside_polygon(geom):  
                self.disconnect()  
                return self._result(False, "El punto debe estar dentro de un poligono", [])  
  
            self.cur.execute(  
                """  
                INSERT INTO campus_puntos  
                (tipo, estado, material, codigo_inventario, observacion, geom)  
            """)
```

```

        VALUES (%s, %s, %s, %s, %s,
                  ST_SnapToGrid(ST_GeomFromText(%s, %s), %s))
    RETURNING id
    """
    [
        d["tipo"],
        d["estado"],
        d["material"],
        d["codigo_inventario"],
        d["observacion"],
        geom,
        EPSG_CODE,
        SNAP_DISTANCE,
    ],
)
row = self.cur.fetchall()
self.conn.commit()
self.disconnect()
return self._result(True, "Inserted", row)
except Exception as e:
    self.conn.rollback()
    self.disconnect()
    return self._result(False, str(e), [])

def select_tuplas(self, d):
    try:
        id_min = int(d.get("id", 0))
        self.cur.execute(
            """
            SELECT id, tipo, estado, material, codigo_inventario, observacion,
                   ST_AsText(geom) AS geom
            FROM campus_puntos
            WHERE id > %s
            ORDER BY id
            """,
            [id_min],
        )
        rows = self.cur.fetchall()
        self.disconnect()
        return self._result(True, "Selected", rows)
    except Exception as e:
        self.disconnect()
        return self._result(False, str(e), [])

def select_diccionarios(self, d):
    try:
        id_min = int(d.get("id", 0))
        self.cur.close()
        self.cur = self.conn.cursor(row_factory=dict_row)
        self.cur.execute(
            """
            SELECT id, tipo, estado, material, codigo_inventario, observacion,
                   ST_AsText(geom) AS geom
            FROM campus_puntos
            WHERE id > %s
            ORDER BY id
            """,
            [id_min],
        )
        rows = self.cur.fetchall()
        self.disconnect()
        return self._result(True, "Selected", rows)
    except Exception as e:
        self.disconnect()
        return self._result(False, str(e), [])

def update(self, d):
    try:
        self._check_keys(
            d,
            ["id", "tipo", "estado", "material", "codigo_inventario", "observacion", "geom"],
        )
        geom = d["geom"]

        if not self._is_valid_geom(geom):
            self.disconnect()
            return self._result(False, "La geometria del punto no es valida", [])

        if not self._point_inside_polygon(geom):
            self.disconnect()

```

```

        return self._result(False, "El punto debe estar dentro de un poligono", [])

    self.cur.execute(
        """
        UPDATE campus_puntos
        SET tipo = %s,
            estado = %s,
            material = %s,
            codigo_inventario = %s,
            observacion = %s,
            geom = ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)
        WHERE id = %s
        """
        ,
        [
            d["tipo"],
            d["estado"],
            d["material"],
            d["codigo_inventario"],
            d["observacion"],
            geom,
            EPSG_CODE,
            SNAP_DISTANCE,
            int(d["id"]),
        ],
    )
    affected = self.cur.rowcount
    self.conn.commit()
    self.disconnect()
    return self._result(True, "Updated", [{"rows": affected}])
except Exception as e:
    self.conn.rollback()
    self.disconnect()
    return self._result(False, str(e), [])

def delete(self, d):
    try:
        self._check_keys(d, ["id"])
        self.cur.execute("DELETE FROM campus_puntos WHERE id = %s", [int(d["id"])])
        affected = self.cur.rowcount
        self.conn.commit()
        self.disconnect()
        return self._result(True, "Deleted", [{"rows": affected}])
    except Exception as e:
        self.conn.rollback()
        self.disconnect()
        return self._result(False, str(e), [])

```

6.4 lineasOOP.py

C:\proyecto_desweb\djangopapi\scripts\p1\psycampus\lineasOOP.py

```

# scripts/p1/psycampus/lineasOOP.py
# CRUD de campus_lineas usando psycopg y SQL directo.
# Regla especial: una linea no debe cruzarse/intersecarse realmente con otra linea.

from psycopg.rows import dict_row
from mylib.connect import connect
from mylib.p1Settings import EPSG_CODE, SNAP_DISTANCE

def parse_bool(value):
    if isinstance(value, bool):
        return value
    return str(value).lower() in ["true", "1", "si", "sí", "yes"]

class LineasOOP:
    def __init__(self):
        self.conn = connect()
        self.cur = self.conn.cursor()

    def disconnect(self):
        self.cur.close()
        self.conn.close()

    def _result(self, ok, message, data):
        return {"ok": ok, "message": message, "data": data}

    def _check_keys(self, d, keys):

```

```

        for key in keys:
            if key not in d:
                raise Exception(f"Falta el campo: {key}")

def _is_valid_geom(self, geom_wkt):
    self.cur.execute(
        """
        SELECT ST_IsValid(
            ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)
        )
        """,
        [geom_wkt, EPSG_CODE, SNAP_DISTANCE],
    )
    return self.cur.fetchone()[0]

def _has_real_intersection(self, geom_wkt, exclude_id=None):
    params = [geom_wkt, EPSG_CODE, SNAP_DISTANCE]
    extra = ""
    if exclude_id is not None:
        extra = "AND id != %s"
        params.append(int(exclude_id))

    self.cur.execute(
        f"""
        SELECT id
        FROM campus_lineas
        WHERE ST_Relate(
            geom,
            ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),
            'T*****'
        )
        {extra}
        """,
        params,
    )
    return self.cur.fetchall()

def insert(self, d):
    try:
        self._check_keys(
            d,
            ["tipo_via", "pavimento", "accesible", "codigo_tramo", "observacion", "geom"],
        )
        geom = d["geom"]

        if not self._is_valid_geom(geom):
            self.disconnect()
            return self._result(False, "La geometria de la linea no es valida", [])

        rows = self._has_real_intersection(geom)
        if len(rows) > 0:
            self.disconnect()
            return self._result(False, "La linea se interseca con otra linea", rows)

        self.cur.execute(
            """
            INSERT INTO campus_lineas
            (tipo_via, pavimento, accesible, codigo_tramo, observacion, geom)
            VALUES (%s, %s, %s, %s, %s,
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s))
            RETURNING id
            """,
            [
                d["tipo_via"],
                d["pavimento"],
                parse_bool(d["accesible"]),
                d["codigo_tramo"],
                d["observacion"],
                geom,
                EPSG_CODE,
                SNAP_DISTANCE,
            ],
        )
        row = self.cur.fetchall()
        self.conn.commit()
        self.disconnect()
        return self._result(True, "Inserted", row)
    except Exception as e:
        self.conn.rollback()
        self.disconnect()

```

```

        return self._result(False, str(e), [])

def select_tuplas(self, d):
    try:
        id_min = int(d.get("id", 0))
        self.cur.execute(
            """
            SELECT id, tipo_via, pavimento, accesible, codigo_tramo, observacion,
                   longitud, ST_AsText(geom) AS geom
            FROM campus_lineas
            WHERE id > %s
            ORDER BY id
            """,
            [id_min],
        )
        rows = self.cur.fetchall()
        self.disconnect()
        return self._result(True, "Selected", rows)
    except Exception as e:
        self.disconnect()
        return self._result(False, str(e), [])

def select_diccionarios(self, d):
    try:
        id_min = int(d.get("id", 0))
        self.cur.close()
        self.cur = self.conn.cursor(row_factory=dict_row)
        self.cur.execute(
            """
            SELECT id, tipo_via, pavimento, accesible, codigo_tramo, observacion,
                   longitud, ST_AsText(geom) AS geom
            FROM campus_lineas
            WHERE id > %s
            ORDER BY id
            """,
            [id_min],
        )
        rows = self.cur.fetchall()
        self.disconnect()
        return self._result(True, "Selected", rows)
    except Exception as e:
        self.disconnect()
        return self._result(False, str(e), [])

def update(self, d):
    try:
        self._check_keys(
            d,
            ["id", "tipo_via", "pavimento", "accesible", "codigo_tramo", "observacion", "geom"],
        )
        geom = d["geom"]

        if not self._is_valid_geom(geom):
            self.disconnect()
            return self._result(False, "La geometria de la linea no es valida", [])

        rows = self._has_real_intersection(geom, exclude_id=d["id"])
        if len(rows) > 0:
            self.disconnect()
            return self._result(False, "La linea se interseca con otra linea", rows)

        self.cur.execute(
            """
            UPDATE campus_lineas
            SET tipo_via = %s,
                pavimento = %s,
                accesible = %s,
                codigo_tramo = %s,
                observacion = %s,
                geom = ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)
            WHERE id = %s
            """,
            [
                d["tipo_via"],
                d["pavimento"],
                parse_bool(d["accesible"]),
                d["codigo_tramo"],
                d["observacion"],
                geom,
                EPSG_CODE,
            ],
        )

```

```

        SNAP_DISTANCE,
        int(d["id"]),
    ],
)
affected = self.cur.rowcount
self.conn.commit()
self.disconnect()
return self._result(True, "Updated", [{"rows": affected}])
except Exception as e:
    self.conn.rollback()
    self.disconnect()
    return self._result(False, str(e), [])

def delete(self, d):
    try:
        self._check_keys(d, ["id"])
        self.cur.execute("DELETE FROM campus_lineas WHERE id = %s", [int(d["id"])])
        affected = self.cur.rowcount
        self.conn.commit()
        self.disconnect()
        return self._result(True, "Deleted", [{"rows": affected}])
    except Exception as e:
        self.conn.rollback()
        self.disconnect()
        return self._result(False, str(e), [])

```

6.5 poligonosOOP.py

C:\proyecto_desweb\djangoapi\scripts\p1\psycampus\poligonosOOP.py

```

# scripts/p1/psycampus/poligonosOOP.py
# CRUD de campus_poligonos usando psycopg y SQL directo.
# Regla espacial: un poligono no debe cruzarse/intersecarse realmente con otro poligono.

from psycopg.rows import dict_row
from myLib.connect import connect
from myLib.p1Settings import EPSG_CODE, SNAP_DISTANCE

class PoligonosOOP:
    def __init__(self):
        self.conn = connect()
        self.cur = self.conn.cursor()

    def disconnect(self):
        self.cur.close()
        self.conn.close()

    def _result(self, ok, message, data):
        return {"ok": ok, "message": message, "data": data}

    def _check_keys(self, d, keys):
        for key in keys:
            if key not in d:
                raise Exception(f"Falta el campo: {key}")

    def _is_valid_geom(self, geom_wkt):
        self.cur.execute(
            """
            SELECT ST_IsValid(
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)
            )
            """,
            [geom_wkt, EPSG_CODE, SNAP_DISTANCE],
        )
        return self.cur.fetchone()[0]

    def _has_real_intersection(self, geom_wkt, exclude_id=None):
        params = [geom_wkt, EPSG_CODE, SNAP_DISTANCE]
        extra = ""
        if exclude_id is not None:
            extra = "AND id != %s"
            params.append(int(exclude_id))

        self.cur.execute(
            f"""
            SELECT id
            FROM campus_poligonos
            WHERE ST_Relate(

```

```

        geom,
        ST_SnapToGrid(ST_GeomFromText(%, %s), %s),
        'T*****'
    )
    {extra}
    """
    params,
)
return self.cur.fetchall()

def insert(self, d):
    try:
        self._check_keys(
            d,
            ["nombre", "uso_principal", "pisos", "estado", "observacion", "geom"],
        )
        geom = d["geom"]

        if not self._is_valid_geom(geom):
            self.disconnect()
            return self._result(False, "La geometria del poligono no es valida", [])

        rows = self._has_real_intersection(geom)
        if len(rows) > 0:
            self.disconnect()
            return self._result(False, "El poligono se interseca con otro poligono", rows)

        self.cur.execute(
            """
            INSERT INTO campus_poligonos
            (nombre, uso_principal, pisos, estado, observacion, geom)
            VALUES (%, %s, %s, %s, %s, %s,
                    ST_SnapToGrid(ST_GeomFromText(%, %s), %s))
            RETURNING id
            """,
            [
                d["nombre"],
                d["uso_principal"],
                int(d["pisos"]) if d["pisos"] not in [None, ""] else None,
                d["estado"],
                d["observacion"],
                geom,
                EPSG_CODE,
                SNAP_DISTANCE,
            ],
        )
        row = self.cur.fetchall()
        self.conn.commit()
        self.disconnect()
        return self._result(True, "Inserted", row)
    except Exception as e:
        self.conn.rollback()
        self.disconnect()
        return self._result(False, str(e), [])

def select_tuplas(self, d):
    try:
        id_min = int(d.get("id", 0))
        self.cur.execute(
            """
            SELECT id, nombre, uso_principal, pisos, estado, observacion,
                   area, perimetro, ST_AsText(geom) AS geom
            FROM campus_poligonos
            WHERE id > %s
            ORDER BY id
            """,
            [id_min],
        )
        rows = self.cur.fetchall()
        self.disconnect()
        return self._result(True, "Selected", rows)
    except Exception as e:
        self.disconnect()
        return self._result(False, str(e), [])

def select_diccionarios(self, d):
    try:
        id_min = int(d.get("id", 0))
        self.cur.close()
        self.cur = self.conn.cursor(row_factory=dict_row)

```

```

        self.cur.execute(
            """
            SELECT id, nombre, uso_principal, pisos, estado, observacion,
                   area, perimetro, ST_AsText(geom) AS geom
            FROM campus_poligonos
            WHERE id > %s
            ORDER BY id
            """,
            [id_min],
        )
        rows = self.cur.fetchall()
        self.disconnect()
        return self._result(True, "Selected", rows)
    except Exception as e:
        self.disconnect()
        return self._result(False, str(e), [])

def update(self, d):
    try:
        self._check_keys(
            d,
            ["id", "nombre", "uso_principal", "pisos", "estado", "observacion", "geom"],
        )
        geom = d["geom"]

        if not self._is_valid_geom(geom):
            self.disconnect()
            return self._result(False, "La geometria del poligono no es valida", [])

        rows = self._has_real_intersection(geom, exclude_id=d["id"])
        if len(rows) > 0:
            self.disconnect()
            return self._result(False, "El poligono se interseca con otro poligono", rows)

        self.cur.execute(
            """
            UPDATE campus_poligonos
            SET nombre = %s,
                uso_principal = %s,
                pisos = %s,
                estado = %s,
                observacion = %s,
                geom = ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)
            WHERE id = %s
            """,
            [
                d["nombre"],
                d["uso_principal"],
                int(d["pisos"]) if d["pisos"] not in [None, ""] else None,
                d["estado"],
                d["observacion"],
                geom,
                EPSG_CODE,
                SNAP_DISTANCE,
                int(d["id"]),
            ],
        )
        affected = self.cur.rowcount
        self.conn.commit()
        self.disconnect()
        return self._result(True, "Updated", [{"rows": affected}])
    except Exception as e:
        self.conn.rollback()
        self.disconnect()
        return self._result(False, str(e), [])

def delete(self, d):
    try:
        self._check_keys(d, ["id"])
        self.cur.execute("DELETE FROM campus_poligonos WHERE id = %s", [int(d["id"])])
        affected = self.cur.rowcount
        self.conn.commit()
        self.disconnect()
        return self._result(True, "Deleted", [{"rows": affected}])
    except Exception as e:
        self.conn.rollback()
        self.disconnect()
        return self._result(False, str(e), [])

```

6.6 main.py

C:\proyecto_desweb\djangoapi\scripts\p1\main.py

```
# scripts/p1/main.py
# Script de prueba para la parte psycopg.
# Se ejecuta desde /usr/src/app/scripts/p1 con:
# python main.py <puntos|lineas|poligonos> <insert|select_tuplas|select_diccionarios|update|delete>

import sys
from psycampus.puntosOOP import PuntosOOP
from psycampus.lineasOOP import LineasOOP
from psycampus.poligonosOOP import PoligonosOOP

def main():
    if len(sys.argv) != 3:
        print("Uso: python main.py <puntos|lineas|poligonos> <insert|select_tuplas|select_diccionarios|update|delete>")
        return

    tabla = sys.argv[1].lower()
    accion = sys.argv[2].lower()

    if accion not in ["insert", "select_tuplas", "select_diccionarios", "update", "delete"]:
        print("Accion no valida")
        return

    if tabla == "poligonos":
        obj = PoligonosOOP()
        data_insert = {
            "nombre": "Edificio A",
            "uso_principal": "Docencia",
            "pisos": 3,
            "estado": "Activo",
            "observacion": "Poligono de prueba",
            "geom": "POLYGON((0 0, 100 0, 100 100, 0 100, 0 0))",
        }
        data_update = {
            "id": 1,
            "nombre": "Edificio A Mod",
            "uso_principal": "Investigacion",
            "pisos": 4,
            "estado": "Activo",
            "observacion": "Poligono actualizado",
            "geom": "POLYGON((0 0, 120 0, 120 100, 0 100, 0 0))",
        }
    elif tabla == "puntos":
        obj = PuntosOOP()
        data_insert = {
            "tipo": "Papelera",
            "estado": "Bueno",
            "material": "Metal",
            "codigo_inventario": "PT001",
            "observacion": "Punto de prueba",
            "geom": "POINT(20 20)",
        }
        data_update = {
            "id": 1,
            "tipo": "Papelera Solar",
            "estado": "Excelente",
            "material": "Metal",
            "codigo_inventario": "PT001",
            "observacion": "Punto actualizado",
            "geom": "POINT(25 25)",
        }
    elif tabla == "lineas":
        obj = LineasOOP()
        data_insert = {
            "tipo_via": "Camino",
            "pavimento": "Asfalto",
            "accesible": True,
            "codigo_tramo": "TR001",
            "observacion": "Linea de prueba",
            "geom": "LINESTRING(10 10, 20 20, 30 25)",
        }
        data_update = {
            "id": 1,
            "tipo_via": "Ciclovia",
```

```

        "pavimento": "Hormigon",
        "accesible": True,
        "codigo_tramo": "TR001",
        "observacion": "Linea actualizada",
        "geom": "LINESTRING(12 12, 22 22, 35 28)",
    }
else:
    print("Tabla no valida")
    return

data_select = {"id": 0}
data_delete = {"id": 1}

if accion == "insert":
    print(obj.insert(data_insert))
elif accion == "select_tuplas":
    print(obj.select_tuplas(data_select))
elif accion == "select_diccionarios":
    print(obj.select_diccionarios(data_select))
elif accion == "update":
    print(obj.update(data_update))
elif accion == "delete":
    print(obj.delete(data_delete))

if __name__ == "__main__":
    main()

```

6.7 Pruebas de psycpg

```

# Attach Shell del contenedor djangoapi
cd /usr/src/app/scripts/p1

# Orden recomendado: primero poligonos, luego puntos, luego lineas
python main.py poligonos insert
python main.py poligonos selectasdict
python main.py puntos insert
python main.py puntos selectasdict
python main.py lineas insert
python main.py lineas selectasdict

# Luego puedes probar update/delete
python main.py poligonos update
python main.py puntos update
python main.py lineas update
python main.py puntos delete
python main.py lineas delete
python main.py poligonos delete

```

Orden obligatorio: El punto necesita un poligono existente para pasar ST_Within. Por eso se inserta primero poligono, luego punto.

7. Parte B - Django con modelos y scripts

Django trabaja con proyecto_desweb. La conexion no se configura en connect.py; se configura en djangoapi/settings.py. Los scripts se ejecutan con manage.py runscript.

7.1 Crear app campus si no existe

```

# Attach Shell del contenedor djangoapi
cd /usr/src/app
python manage.py startapp campus

```

7.2 settings.py

No reemplaces todo settings.py si tu profesor ya tiene configuraciones adicionales. Verifica que existan estas apps y que DATABASES apunte a proyecto_desweb con motor PostGIS.

```

# Fragmento de djangoapi/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",

```

```

    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "django.contrib.gis",
    "django_extensions",

    "campus",
]

# Fragmento orientativo de DATABASES en .env.dev
DATABASES = {
    "default": {
        "ENGINE": "django.contrib.gis.db.backends.postgis",
        "NAME": "proyecto_desweb",
        "USER": "postgres",
        "PASSWORD": "postgres",
        "HOST": "postgis",
        "PORT": "5432",
    }
}

```

7.3 models.py

C:\proyecto_desweb\djangoapi\campus\models.py

```

# campus/models.py
# Modelos Django para las tres capas espaciales.
# Django usara estos modelos para crear o leer las tablas de proyecto_desweb.

from django.contrib.gis.db import models

class Punto(models.Model):
    tipo = models.CharField(max_length=50)
    estado = models.CharField(max_length=50)
    material = models.CharField(max_length=50, blank=True, null=True)
    codigo_inventario = models.CharField(max_length=20, unique=True)
    observacion = models.TextField(blank=True, null=True)
    geom = models.PointField(srid=25830)

    class Meta:
        db_table = "campus_puntos"

    def __str__(self):
        return f"Punto {self.id} - {self.tipo}"

class Linea(models.Model):
    tipo_via = models.CharField(max_length=50)
    pavimento = models.CharField(max_length=50, blank=True, null=True)
    accesible = models.BooleanField(default=True)
    codigo_tramo = models.CharField(max_length=20, unique=True)
    observacion = models.TextField(blank=True, null=True)
    geom = models.LineStringField(srid=25830)
    longitud = models.FloatField(blank=True, null=True, editable=False)

    class Meta:
        db_table = "campus_lineas"

    def save(self, *args, **kwargs):
        # EPSG:25830 esta en metros, por eso geom.length devuelve longitud aproximada en metros.
        if self.geom:
            self.longitud = self.geom.length
        super().save(*args, **kwargs)

    def __str__(self):
        return f"Linea {self.id} - {self.codigo_tramo}"

class Poligono(models.Model):
    nombre = models.CharField(max_length=100)
    uso_principal = models.CharField(max_length=50, blank=True, null=True)
    pisos = models.IntegerField(blank=True, null=True)
    estado = models.CharField(max_length=50, blank=True, null=True)
    observacion = models.TextField(blank=True, null=True)
    geom = models.PolygonField(srid=25830)
    area = models.FloatField(blank=True, null=True, editable=False)

```

```

perimetro = models.FloatField(blank=True, null=True, editable=False)

class Meta:
    db_table = "campus_poligonos"

def save(self, *args, **kwargs):
    # EPSG:25830 esta en metros; area en m2 y perimetro en m.
    if self.geom:
        self.area = self.geom.area
        self.perimetro = self.geom.length
    super().save(*args, **kwargs)

def __str__(self):
    return f"Poligono {self.id} - {self.nombre}"

```

7.4 admin.py

C:\proyecto_desweb\djangoapi\campus\admin.py

```

# campus/admin.py
# Registra los modelos para poder revisarlos desde el panel de administracion de Django.

from django.contrib import admin
from campus.models import Punto, Linea, Poligono

admin.site.register(Punto)
admin.site.register(Linea)
admin.site.register(Poligono)

```

7.5 Migraciones desde cero

```

# Attach Shell del contenedor djangoapi
cd /usr/src/app
python manage.py check
python manage.py makemigrations campus
python manage.py migrate
python manage.py showmigrations campus

```

7.6 Si las tablas ya existen en proyecto_desweb

```

# Si pgAdmin ya muestra campus_puntos, campus_lineas y campus_poligonos:
cd /usr/src/app
python manage.py check
python manage.py showmigrations campus

# Si las tablas existen pero la migracion aparece pendiente [ ]:
python manage.py migrate campus --fake-initial

# No borres proyecto_desweb salvo que quieras reiniciar completamente.

```

7.7 insertdj.py

C:\proyecto_desweb\djangoapi\scripts\p1\djcampus\insertdj.py

```

# scripts/p1/djcampus/insertdj.py
# Inserciones con Django ORM + validaciones espaciales en PostGIS.
# Se ejecuta con: python manage.py runscript scripts.p1.djcampus.insertdj --script-args poligonos

from django.contrib.gis.geos import GEOSGeometry
from django.forms.models import model_to_dict
from django.db import connection
from campus.models import Punto, Linea, Poligono
from scripts.p1.myLib.p1Settings import EPSG_CODE, SNAP_DISTANCE

def parse_bool(value):
    if isinstance(value, bool):
        return value
    return str(value).lower() in ["true", "1", "si", "sí", "yes"]

def snap_geom(geom_wkt):
    # PostGIS redondea la geometria. Luego Django la recibe como GEOSGeometry.
    cur = connection.cursor()
    cur.execute(
        """

```

```

        SELECT ST_AsText(
            ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)
        )
        """
        [geom_wkt, EPSG_CODE, SNAP_DISTANCE],
    )
    snapped_wkt = cur.fetchone()[0]
    return GEOSGeometry(snapped_wkt, srid=EPSG_CODE)

def model_to_dict_with_geom(obj):
    # model_to_dict no devuelve la geometria como texto WKT; por eso la convertimos.
    d = model_to_dict(obj)
    d["geom"] = obj.geom.wkt
    return d

def insert_punto(d):
    try:
        g = snap_geom(d["geom"])
        if not g.valid:
            return {"ok": False, "message": "Geometria de punto invalida", "data": []}

        # El punto debe estar dentro de un poligono existente.
        cur = connection.cursor()
        cur.execute(
            """
            SELECT id
            FROM campus_poligonos
            WHERE ST_Within(
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),
                geom
            )
            LIMIT 1
            """
            ,
            [d["geom"], EPSG_CODE, SNAP_DISTANCE],
        )
        if len(cur.fetchall()) == 0:
            return {"ok": False, "message": "El punto debe estar dentro de un poligono", "data": []}

        obj = Punto(
            tipo=d["tipo"],
            estado=d["estado"],
            material=d.get("material"),
            codigo_inventario=d["codigo_inventario"],
            observacion=d.get("observacion"),
            geom=g,
        )
        obj.save()
        return {"ok": True, "message": "Punto insertado", "data": [model_to_dict_with_geom(obj)]}
    except Exception as e:
        return {"ok": False, "message": str(e), "data": []}

def insert_linea(d):
    try:
        g = snap_geom(d["geom"])
        if not g.valid:
            return {"ok": False, "message": "Geometria de linea invalida", "data": []}

        # Evita cruces reales con otras lineas.
        cur = connection.cursor()
        cur.execute(
            """
            SELECT id
            FROM campus_lineas
            WHERE ST_Relate(
                geom,
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),
                'T*****'
            )
            """
            ,
            [d["geom"], EPSG_CODE, SNAP_DISTANCE],
        )
        rows = cur.fetchall()
        if len(rows) > 0:
            return {"ok": False, "message": "La linea se interseca con otra linea", "data": rows}

        obj = Linea(
            tipo_via=d["tipo_via"],

```

```

        pavimento=d.get("pavimento"),
        accesible=parse_bool(d.get("accesible", True)),
        codigo_tramo=d["codigo_tramo"],
        observacion=d.get("observacion"),
        geom=g,
    )
    obj.save()
    return {"ok": True, "message": "Linea insertada", "data": [model_to_dict_with_geom(obj)]}
except Exception as e:
    return {"ok": False, "message": str(e), "data": []}

def insert_poligono(d):
    try:
        g = snap_geom(d["geom"])
        if not g.valid:
            return {"ok": False, "message": "Geometria de poligono invalida", "data": []}

        # Evita cruces reales con otros poligonos.
        cur = connection.cursor()
        cur.execute(
            """
            SELECT id
            FROM campus_poligonos
            WHERE ST_Relate(
                geom,
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),
                'T*****'
            )
            """
            , [d["geom"], EPSG_CODE, SNAP_DISTANCE],
        )
        rows = cur.fetchall()
        if len(rows) > 0:
            return {"ok": False, "message": "El poligono se interseca con otro poligono", "data": rows}

        obj = Poligono(
            nombre=d["nombre"],
            uso_principal=d.get("uso_principal"),
            pisos=int(d["pisos"]) if d.get("pisos") not in [None, ""] else None,
            estado=d.get("estado"),
            observacion=d.get("observacion"),
            geom=g,
        )
        obj.save()
        return {"ok": True, "message": "Poligono insertado", "data": [model_to_dict_with_geom(obj)]}
    except Exception as e:
        return {"ok": False, "message": str(e), "data": []}

def run(*args):
    if len(args) < 1:
        print("Uso: --script-args puntos|lineas|poligonos")
        return

    tabla = args[0].lower()

    if tabla == "poligonos":
        data = {
            "nombre": "Edificio A",
            "uso_principal": "Docencia",
            "pisos": 3,
            "estado": "Activo",
            "observacion": "Poligono de prueba",
            "geom": "POLYGON((0 0, 100 0, 100 100, 0 100, 0 0))",
        }
        print(insert_poligono(data))
    elif tabla == "puntos":
        data = {
            "tipo": "Papelera",
            "estado": "Bueno",
            "material": "Metal",
            "codigo_inventario": "PT001",
            "observacion": "Punto de prueba",
            "geom": "POINT(20 20)",
        }
        print(insert_punto(data))
    elif tabla == "lineas":
        data = {
            "tipo_via": "Camino",

```

```

        "pavimento": "Asfalto",
        "accesible": True,
        "codigo_tramo": "TR001",
        "observacion": "Linea de prueba",
        "geom": "LINESTRING(10 10, 20 20, 30 25)",
    }
    print(insert_linea(data))
else:
    print("Tabla no valida")

```

7.8 selectdj.py

C:\proyecto_desweb\djangopapi\scripts\p1\djcampus\selectdj.py

```

# scripts/p1/djcampus/selectdj.py
# Consultas con Django ORM para puntos, lineas y poligonos.
# Se ejecuta con: python manage.py runscript scripts.p1.djcampus.selectdj --script-args poligonos 1

from django.forms.models import model_to_dict
from campus.models import Punto, Linea, Poligono

def object_to_dict(obj):
    d = model_to_dict(obj)
    d["geom"] = obj.geom.wkt
    return d

def select_data(model, id_value):
    # id_value = 0 devuelve todos los registros.
    # id_value > 0 devuelve solo el registro con ese id.
    if id_value == 0:
        lista = list(model.objects.filter(id__gt=0).order_by("id"))
    else:
        lista = list(model.objects.filter(id=id_value))

    if len(lista) == 0:
        return {"ok": False, "message": "No se encontraron registros", "data": []}

    data = [object_to_dict(obj) for obj in lista]
    return {"ok": True, "message": "Datos recuperados", "data": data}

def run(*args):
    if len(args) < 1:
        print("Uso: --script-args puntos|lineas|poligonos [id]")
        return

    tabla = args[0].lower()
    id_buscado = int(args[1]) if len(args) > 1 else 0

    tablas = {
        "puntos": Punto,
        "lineas": Linea,
        "poligonos": Poligono,
    }

    if tabla in tablas:
        print(select_data(tablas[tabla], id_buscado))
    else:
        print("Tabla no valida")

```

7.9 updatedj.py

C:\proyecto_desweb\djangopapi\scripts\p1\djcampus\updatedj.py

```

# scripts/p1/djcampus/updatedj.py
# Actualizaciones con Django ORM + validaciones espaciales en PostGIS.
# Se ejecuta con: python manage.py runscript scripts.p1.djcampus.updatedj --script-args poligonos 1

from django.contrib.gis.geos import GEOSGeometry
from django.forms.models import model_to_dict
from django.db import connection
from campus.models import Punto, Linea, Poligono
from scripts.p1.myLib.p1Settings import EPSG_CODE, SNAP_DISTANCE

def parse_bool(value):

```

```

if isinstance(value, bool):
    return value
return str(value).lower() in ["true", "1", "si", "sí", "yes"]

def snap_geom(geom_wkt):
    cur = connection.cursor()
    cur.execute(
        """
        SELECT ST_AsText(
            ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)
        )
        """,
        [geom_wkt, EPSG_CODE, SNAP_DISTANCE],
    )
    snapped_wkt = cur.fetchone()[0]
    return GEOSGeometry(snapped_wkt, srid=EPSG_CODE)

def model_to_dict_with_geom(obj):
    d = model_to_dict(obj)
    d["geom"] = obj.geom.wkt
    return d

def update_punto(d):
    try:
        g = snap_geom(d["geom"])
        if not g.valid:
            return {"ok": False, "message": "Geometria de punto invalida", "data": []}

        cur = connection.cursor()
        cur.execute(
            """
            SELECT id
            FROM campus_poligonos
            WHERE ST_Within(
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),
                geom
            )
            LIMIT 1
            """,
            [d["geom"], EPSG_CODE, SNAP_DISTANCE],
        )
        if len(cur.fetchall()) == 0:
            return {"ok": False, "message": "El punto debe estar dentro de un poligono", "data": []}

        qs = Punto.objects.filter(id=d["id"])
        if not qs.exists():
            return {"ok": False, "message": "No existe el punto", "data": []}

        obj = qs[0]
        obj.tipo = d["tipo"]
        obj.estado = d["estado"]
        obj.material = d.get("material")
        obj.codigo_inventario = d["codigo_inventario"]
        obj.observacion = d.get("observacion")
        obj.geom = g
        obj.save()
        return {"ok": True, "message": "Punto actualizado", "data": [model_to_dict_with_geom(obj)]}
    except Exception as e:
        return {"ok": False, "message": str(e), "data": []}

def update_linea(d):
    try:
        g = snap_geom(d["geom"])
        if not g.valid:
            return {"ok": False, "message": "Geometria de linea invalida", "data": []}

        cur = connection.cursor()
        cur.execute(
            """
            SELECT id
            FROM campus_lineas
            WHERE ST_Relate(
                geom,
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),
                'T*****'
            )
            """
        )

```

```

        AND id != %s
        """
        [d["geom"], EPSG_CODE, SNAP_DISTANCE, d["id"]],
    )
    rows = cur.fetchall()
    if len(rows) > 0:
        return {"ok": False, "message": "La linea se interseca con otra linea", "data": rows}

    qs = Linea.objects.filter(id=d["id"])
    if not qs.exists():
        return {"ok": False, "message": "No existe la linea", "data": []}

    obj = qs[0]
    obj.tipo_via = d["tipo_via"]
    obj.pavimento = d.get("pavimento")
    obj.accesible = parse_bool(d.get("accesible", True))
    obj.codigo_tramo = d["codigo_tramo"]
    obj.observacion = d.get("observacion")
    obj.geom = g
    obj.save()
    return {"ok": True, "message": "Linea actualizada", "data": [model_to_dict_with_geom(obj)]}
except Exception as e:
    return {"ok": False, "message": str(e), "data": []}

def update_poligono(d):
    try:
        g = snap_geom(d["geom"])
        if not g.valid:
            return {"ok": False, "message": "Geometria de poligono invalida", "data": []}

        cur = connection.cursor()
        cur.execute(
            """
            SELECT id
            FROM campus_poligonos
            WHERE ST_Relate(
                geom,
                ST_SnapToGrid(ST_GeomFromText(%s, %s), %s),
                'T*****'
            )
            AND id != %s
            """
            [d["geom"], EPSG_CODE, SNAP_DISTANCE, d["id"]],
        )
        rows = cur.fetchall()
        if len(rows) > 0:
            return {"ok": False, "message": "El poligono se interseca con otro poligono", "data": rows}

        qs = Poligono.objects.filter(id=d["id"])
        if not qs.exists():
            return {"ok": False, "message": "No existe el poligono", "data": []}

        obj = qs[0]
        obj.nombre = d["nombre"]
        obj.uso_principal = d.get("uso_principal")
        obj.pisos = int(d["pisos"]) if d.get("pisos") not in [None, ""] else None
        obj.estado = d.get("estado")
        obj.observacion = d.get("observacion")
        obj.geom = g
        obj.save()
        return {"ok": True, "message": "Poligono actualizado", "data": [model_to_dict_with_geom(obj)]}
    except Exception as e:
        return {"ok": False, "message": str(e), "data": []}

def run(*args):
    if len(args) < 1:
        print("Uso: --script-args puntos|lineas|poligonos [id]")
        return

    tabla = args[0].lower()
    id_val = int(args[1]) if len(args) > 1 else 1

    if tabla == "puntos":
        data = {
            "id": id_val,
            "tipo": "Papelera Solar",
            "estado": "Excelente",
            "material": "Metal",

```

```

        "codigo_inventario": "PT001",
        "observacion": "Punto actualizado",
        "geom": "POINT(25 25)",
    }
    print(update_punto(data))
elif tabla == "lineas":
    data = {
        "id": id_val,
        "tipo_via": "Ciclovia",
        "pavimento": "Hormigon",
        "accesible": True,
        "codigo_tramo": "TR001",
        "observacion": "Linea actualizada",
        "geom": "LINESTRING(12 12, 22 22, 35 28)",
    }
    print(update_linea(data))
elif tabla == "poligonos":
    data = {
        "id": id_val,
        "nombre": "Edificio A Mod",
        "uso_principal": "Investigacion",
        "pisos": 4,
        "estado": "Activo",
        "observacion": "Poligono actualizado",
        "geom": "POLYGON((0 0, 120 0, 120 100, 0 100, 0 0))",
    }
    print(update_poligono(data))
else:
    print("Tabla no valida")

```

7.10 deletedj.py

C:\proyecto_desweb\djangoapi\scripts\p1\djcampus\deletedj.py

```

# scripts/p1/djcampus/deletedj.py
# Eliminaciones con Django ORM.
# Se ejecuta con: python manage.py runscript scripts.p1.djcampus.deletedj --script-args puntos 1

from campus.models import Punto, Linea, Poligono

def delete_data(model, id_value):
    qs = model.objects.filter(id=id_value)
    lista = list(qs)

    if len(lista) < 1:
        return {"ok": False, "message": f"No se encontro el registro con ID {id_value}", "data": []}

    obj = lista[0]
    obj.delete()
    return {"ok": True, "message": "Registro eliminado con exito", "data": [{"id": id_value}]}

def run(*args):
    if len(args) < 1:
        print("Uso: --script-args puntos|lineas|poligonos [id]")
        return

    tabla = args[0].lower()
    id_a_borrar = int(args[1]) if len(args) > 1 else 1

    tablas = {
        "puntos": Punto,
        "lineas": Linea,
        "poligonos": Poligono,
    }

    if tabla in tablas:
        print(delete_data(tablas[tabla], id_a_borrar))
    else:
        print("Tabla no valida")

```

7.11 Pruebas de scripts Django

```

# Attach Shell del contenedor djangoapi
cd /usr/src/app

python manage.py check

```

```
# Insertar en orden
python manage.py runscript scripts.p1.djcampus.insertdj --script-args poligonos
python manage.py runscript scripts.p1.djcampus.insertdj --script-args puntos
python manage.py runscript scripts.p1.djcampus.insertdj --script-args lineas

# Consultar
python manage.py runscript scripts.p1.djcampus.selectdj --script-args poligonos 1
python manage.py runscript scripts.p1.djcampus.selectdj --script-args puntos 1
python manage.py runscript scripts.p1.djcampus.selectdj --script-args lineas 1

# Actualizar
python manage.py runscript scripts.p1.djcampus.updatedj --script-args poligonos 1
python manage.py runscript scripts.p1.djcampus.updatedj --script-args puntos 1
python manage.py runscript scripts.p1.djcampus.updatedj --script-args lineas 1

# Eliminar si quieres limpiar
python manage.py runscript scripts.p1.djcampus.deletedj --script-args puntos 1
python manage.py runscript scripts.p1.djcampus.deletedj --script-args lineas 1
python manage.py runscript scripts.p1.djcampus.deletedj --script-args poligonos 1
```

8. Parte C - URLs y views

Solo crea esta parte cuando los scripts Django funcionen por consola. Las vistas no deben reemplazar la logica; solo conectan peticiones HTTP con funciones ya probadas.

8.1 campus/urls.py

C:\proyecto_desweb\djangoapi\campus\urls.py

```
# campus/urls.py
# Rutas propias de la app campus.
# Django entrara aqui desde djangoapi/urls.py cuando la URL empiece por /campus/.

from django.urls import path
from . import views

urlpatterns = [
    path("hello/", views.HelloCampus.as_view(), name="hello_campus"),
    path("<str:tabla>/<str:accion>/", views.CampusView.as_view(), name="campus_accion"),
    path("<str:tabla>/<str:accion>/<int:id>/", views.CampusView.as_view(), name="campus_accion_id"),
]
```

8.2 campus/views.py

C:\proyecto_desweb\djangoapi\campus\views.py

```
# campus/views.py
# Capa web de la app campus.
# Recibe peticiones HTTP y llama a las funciones ya probadas en djcampus.

import json
from django.http import JsonResponse
from django.views import View
from django.views.decorators.csrf import csrf_exempt
from django.utils.decorators import method_decorator

from campus.models import Punto, Linea, Poligono
from scripts.p1.djcampus.insertdj import insert_punto, insert_linea, insert_poligono
from scripts.p1.djcampus.selectdj import select_data
from scripts.p1.djcampus.updatedj import update_punto, update_linea, update_poligono
from scripts.p1.djcampus.deletedj import delete_data

MODELOS = {
    "puntos": Punto,
    "lineas": Linea,
    "poligonos": Poligono,
}

INSERTS = {
    "puntos": insert_punto,
    "lineas": insert_linea,
    "poligonos": insert_poligono,
}
```

```

UPDATES = {
    "puntos": update_punto,
    "lineas": update_linea,
    "poligonos": update_poligono,
}

def get_request_data(request):
    # Permite enviar datos como JSON o como form-data/x-www-form-urlencoded.
    if request.content_type and request.content_type.startswith("application/json"):
        try:
            return json.loads(request.body.decode("utf-8"))
        except Exception:
            return {}
    return request.POST.dict()

def normalizar_datos(tabla, data):
    # Ajusta tipos que suelen llegar como texto desde Postman o formularios.
    if tabla == "lineas" and "accesible" in data:
        data["accesible"] = str(data["accesible"]).lower() in ["true", "1", "si", "sí", "yes"]

    if tabla == "poligonos" and "pisos" in data:
        if data["pisos"] not in [None, ""]:
            data["pisos"] = int(data["pisos"])

    return data

@method_decorator(csrf_exempt, name="dispatch")
class HelloCampus(View):
    def get(self, request):
        return JsonResponse({
            "ok": True,
            "message": "Campus. Hello world",
            "data": [request.GET.dict()],
        })

    def post(self, request):
        return JsonResponse({
            "ok": True,
            "message": "Campus. Hello world",
            "data": [get_request_data(request)],
        })

@method_decorator(csrf_exempt, name="dispatch")
class CampusView(View):
    def get(self, request, tabla, accion, id=None):
        if tabla not in MODELOS:
            return JsonResponse({"ok": False, "message": "Tabla no valida", "data": []}, status=400)

        if accion != "select":
            return JsonResponse({"ok": False, "message": "Accion GET no valida. Usa select.", "data": []},
                                status=400)

        id_value = id if id is not None else 0
        resultado = select_data(MODELOS[tabla], id_value)
        return JsonResponse(resultado)

    def post(self, request, tabla, accion, id=None):
        if tabla not in MODELOS:
            return JsonResponse({"ok": False, "message": "Tabla no valida", "data": []}, status=400)

        data = normalizar_datos(tabla, get_request_data(request))

        if accion == "insert":
            resultado = INSERTS[tabla](data)
            return JsonResponse(resultado)

        if accion == "update":
            if id is None:
                return JsonResponse({"ok": False, "message": "Falta id en la URL", "data": []}, status=400)
            data["id"] = id
            resultado = UPDATES[tabla](data)
            return JsonResponse(resultado)

        if accion == "delete":
            if id is None:

```

```

        return JsonResponse({"ok": False, "message": "Falta id en la URL", "data": []}, status=400)
    resultado = delete_data(MODELOS[tabla], id)
    return JsonResponse(resultado)

return JsonResponse({"ok": False, "message": "Accion POST no valida", "data": []}, status=400)

```

8.3 djangoapi/urls.py

C:\proyecto_desweb\djangoapi\djangoapi\urls.py

```

# djangoapi/urls.py
# Archivo principal de rutas del proyecto.
# Si tu profesor ya tiene rutas core/codelist/buildings, conservalas y solo agrega campus.

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),

    # Mantener otras apps si ya existen en tu proyecto:
    # path("core/", include("core.urls")),
    # path("codelist/", include("codelist.urls")),

    path("campus/", include("campus.urls")),
]

```

Importante: Si `djangoapi/urls.py` ya incluye `core`, `codelist` u otras apps del profesor, no las borres. Solo agrega `path("campus/", include("campus.urls"))`.

9. Pruebas por consola, navegador y Postman

9.1 Comprobacion base

```

cd /usr/src/app
python manage.py check
python manage.py showmigrations campus

```

9.2 Pruebas GET en navegador

```

http://127.0.0.1:8001/campus/hello/
http://127.0.0.1:8001/campus/poligonos/select/
http://127.0.0.1:8001/campus/puntos/select/
http://127.0.0.1:8001/campus/lineas/select/
http://127.0.0.1:8001/campus/poligonos/select/1/

```

9.3 Pruebas POST en Postman

```

# Insertar poligono
POST http://127.0.0.1:8001/campus/poligonos/insert/
Content-Type: application/json

{
  "nombre": "Edificio B",
  "uso_principal": "Docencia",
  "pisos": 2,
  "estado": "Activo",
  "observacion": "Insertado desde Postman",
  "geom": "POLYGON((200 0, 300 0, 300 100, 200 100, 200 0))"
}

# Insertar punto dentro del poligono anterior
POST http://127.0.0.1:8001/campus/puntos/insert/
Content-Type: application/json

{
  "tipo": "Banco",
  "estado": "Bueno",
  "material": "Madera",
  "codigo_inventario": "PT002",
  "observacion": "Punto desde Postman",
  "geom": "POINT(220 20)"
}

```

```

}

# Insertar linea
POST http://127.0.0.1:8001/campus/lineas/insert/
Content-Type: application/json

{
  "tipo_via": "Sendero",
  "pavimento": "Tierra",
  "accesible": true,
  "codigo_tramo": "TR002",
  "observacion": "Linea desde Postman",
  "geom": "LINESTRING(210 10, 230 30, 260 40)"
}

```

10. Que hacer si ya esta generado

Si ya tienes tablas en pgAdmin, no empieces borrando. Primero diagnostica si Django esta sincronizado con esas tablas.

Situacion	Que hacer
campus_* existe y showmigrations muestra [X]	No recrees la base. Continua con scripts, urls.py y views.py.
campus_* existe pero showmigrations muestra []	Usa python manage.py migrate campus --fake-initial.
campus_* no existe y campus esta limpio	Ejecuta makemigrations campus y migrate.
Error por campo no nulo en migraciones	Si estas iniciando desde cero, borra migraciones antiguas de campus excepto __init__.py y recrea la base.
proyecto_psy esta vacia	Ejecuta 01_schema.sql en proyecto_psy. Es normal si aun no hiciste psycopg.

```

# Diagnostico minimo
cd /usr/src/app
python manage.py check
python manage.py showmigrations campus

# Si las tablas ya existen pero la migracion esta pendiente:
python manage.py migrate campus --fake-initial

```

11. Relacion con scripts antiguos de scriptsBase.rar

Los scripts antiguos sirven como referencia conceptual, pero conviene consolidarlos en una estructura limpia. La idea es no perder lo aprendido, sino ordenar nombres, imports y responsabilidades.

Archivo antiguo	Uso probable	Que se hace en la version final
psycampus/insert.py, select.py, update.py, delete.py	CRUD procedural separado por operacion.	Se consolida en clases OOP por geometria: puntosOOP.py, lineasOOP.py, poligonosOOP.py.
psycampus/puntos.py, lineas.py, poligonos.py	Logica por tabla, pero puede tener nombres de tablas inconsistentes.	Se mantiene la idea por tabla, pero con nombres correctos: campus_puntos, campus_lineas, campus_poligonos.
djcampus/insertdj.py, selectdj.py, updatedj.py, deletedj.py	Scripts Django principales.	Se reemplazan por versiones completas para puntos, lineas y poligonos.
djcampus/* - copia.py	Copias probablemente mas completas durante la practica.	Se usan como referencia de logica, pero no deben quedar como archivos activos.
djcampus/puntos.py, lineas.py, poligonos.py	Archivos por geometria; algunos suelen venir de plantilla.	Si importan tu_app.models, se descartan o se corrigen a campus.models.
main.py	Entrada principal de pruebas.	Se conserva como entrada oficial para psycopg.
mainDjango.py, mainOOP.py	Entradas alternativas de prueba.	Se mueven a respaldo; el flujo oficial queda documentado en main.py y manage.py runscript.

Criterio de limpieza: Un archivo antiguo se conserva solo si respeta tres reglas: importa `campus.models`, usa tablas `campus_*` y forma parte del flujo oficial de pruebas.

12. Errores frecuentes

Error	Causa probable	Solucion
relation "campus_punto" does not exist	Nombre de tabla mal escrito o script antiguo.	Usar <code>campus_puntos</code> .
relation "lineas" does not exist	Script antiguo sin prefijo <code>campus_</code> .	Usar <code>campus_lineas</code> .
relation "poligonos" does not exist	Script antiguo sin prefijo <code>campus_</code> .	Usar <code>campus_poligonos</code> .
ModuleNotFoundError: tu_app	Archivo copiado de plantilla sin adaptar.	Cambiar a <code>from campus.models import Punto, Linea, Poligono</code> .
ModuleNotFoundError: buildings2.urls	<code>djangoapi/urls.py</code> incluye una app no completa.	Quitar <code>include("buildings2.urls")</code> o usar <code>campus.urls</code> .
<code>bool("False")</code> se guarda como <code>True</code>	Conversion booleana incorrecta.	Usar <code>parse_bool</code> .
duplicate key value violates unique constraint	Repetiste PT001 o TR001.	Cambiar codigo o borrar registro previo.
El punto debe estar dentro de un poligono	Insertaste punto antes que poligono o punto fuera del poligono.	Insertar poligono primero y usar coordenadas internas.
The view did not return an <code>HttpResponse</code>	Alguna rama de <code>views.py</code> no retorna <code>JsonResponse</code> .	Asegurar return en todos los <code>if/else</code> .
CSRF verification failed	Postman envia POST sin token CSRF.	Usar <code>csrf_exempt</code> en las vistas de practica.
connection refused localhost	Desde Docker se intento conectar a localhost.	Usar <code>POSTGRES_HOST=postgis</code> .

13. Comandos finales y checklist

13.1 Secuencia completa desde cero

```
# 1. Windows PowerShell
cd C:\proyecto_desweb
docker compose up

# 2. Otra terminal PowerShell
docker ps

# 3. Attach Shell del contenedor postgis
docker exec -it NOMBRE_CONTENEDOR_POSTGIS /bin/sh
psql -U postgres

-- Dentro de psql
CREATE DATABASE proyecto_psy;
CREATE DATABASE proyecto_desweb;
\c proyecto_psy
CREATE EXTENSION IF NOT EXISTS postgis;
\c proyecto_desweb
CREATE EXTENSION IF NOT EXISTS postgis;
\q

# 4. Attach Shell del contenedor djangoapi
docker exec -it NOMBRE_CONTENEDOR_DJANGO /bin/sh
cd /usr/src/app

# 5. Crear tablas de psycopg
psql -h postgis -U postgres -d proyecto_psy -f /usr/src/app/scripts/p1/01_schema.sql

# 6. Django
python manage.py check
python manage.py makemigrations campus
python manage.py migrate
python manage.py showmigrations campus

# 7. Probar psycopg
cd /usr/src/app/scripts/p1
python main.py poligonos insert
python main.py puntos insert
python main.py lineas insert
```

```
# 8. Probar Django scripts
cd /usr/src/app
python manage.py runscript scripts.p1.djcampus.insertdj --script-args poligonos
python manage.py runscript scripts.p1.djcampus.insertdj --script-args puntos
python manage.py runscript scripts.p1.djcampus.insertdj --script-args lineas
python manage.py runscript scripts.p1.djcampus.selectdj --script-args poligonos 1

# 9. Probar web
# http://127.0.0.1:8001/campus/hello/
# http://127.0.0.1:8001/campus/poligonos/select/

# 10. Cerrar al terminar
exit
docker compose down
```

13.2 Checklist final

- La carpeta p1 no contiene archivos mezclados de buildings como código activo.
- myLib/p1Settings.py apunta a proyecto_psy y usa POSTGRES_HOST=postgis como valor por defecto.
- proyecto_psy tiene campus_puntos, campus_lineas y campus_poligonos.
- proyecto_desweb tiene campus_puntos, campus_lineas, campus_poligonos y tablas django_*/auth_*.
- campus/models.py usa db_table correcto: campus_puntos, campus_lineas, campus_poligonos.
- python manage.py check no devuelve errores.
- runscript funciona para poligonos, puntos y lineas.
- campus/urls.py existe.
- campus/views.py devuelve JsonResponse en todos los casos.
- djangoapi/urls.py incluye path("campus/", include("campus.urls")).
- Los endpoints /campus/hello/ y /campus/poligonos/select/ responden.

Cierre: El criterio para avanzar a `urls.py` y `views.py` es simple: primero deben funcionar los scripts por consola. Si la consola funciona, las vistas solo conectan HTTP con funciones ya probadas.
